Kinvecta

A KINVECTA SERIES

How Large Language Models Work

From math to transformers

EPISODE 04 · PROBABILITY, E

Kinvecta

HOW LARGE LANGUAGE MODELS WORK · EPISODE 4

Two models. Which one is more sure?

Both have a tallest bar. Today “how sure” becomes a number you can compute by hand.

19-minute lesson

Companion notebook



MARK

WHY THIS MATTERS

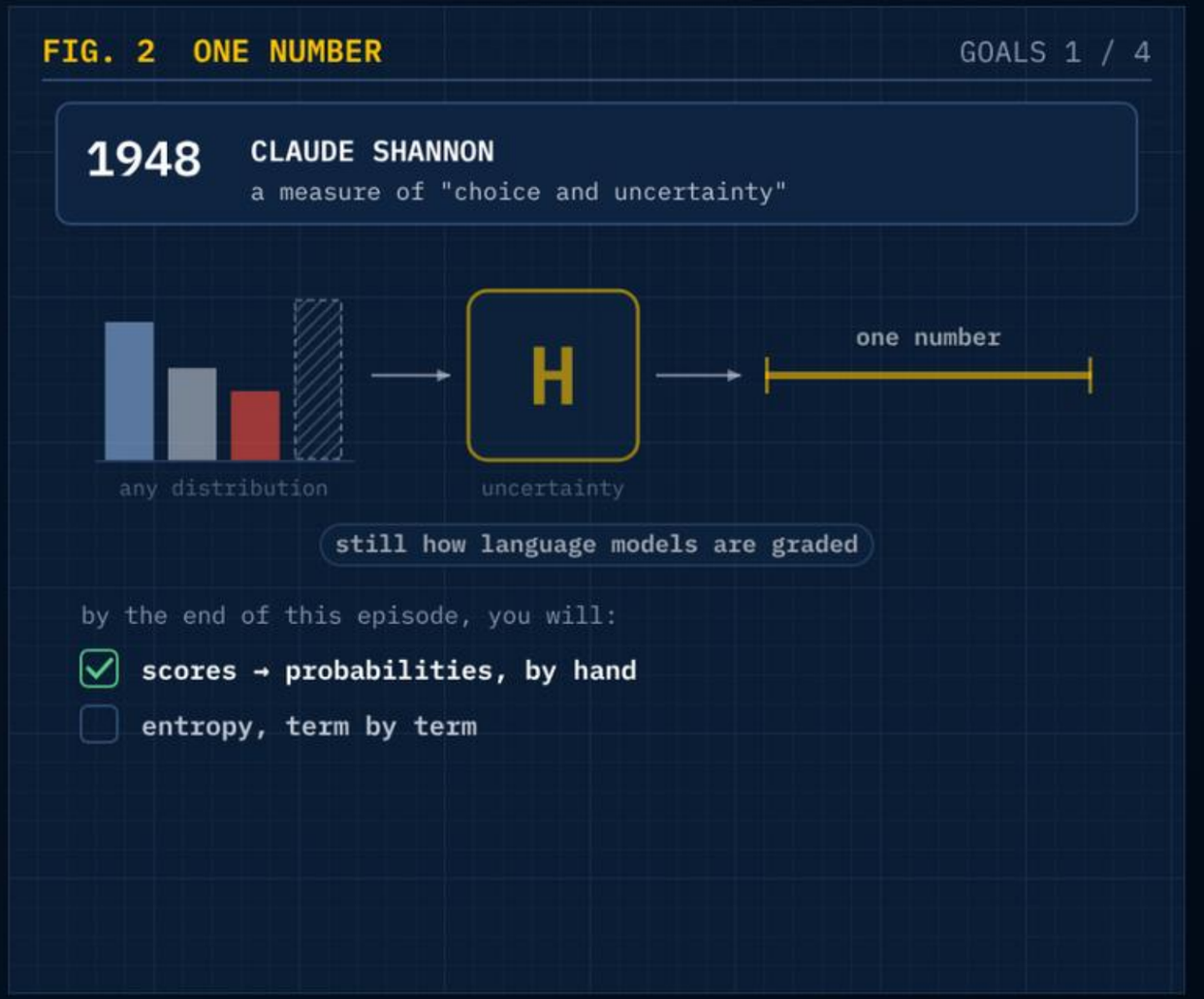
One number for uncertainty

Shannon, 1948: a single value for any probability distribution, built to measure uncertainty.

01 Language models are still graded with it

02 Turn raw scores into probabilities by hand

03 Compute entropy term by term



Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal* 27 (1948). Shannon, "Prediction and Entropy of Printed English," *BSTJ* (1951).

EPISODE 4 · PROBABILITY, ENTROPY, PERPLEXITY

Probability, entropy, perplexity

Four parts, one sentence, and a notebook with every number.

01 Probability and the chain rule

02 Softmax: scores to probabilities

03 Entropy: average surprise



Companion notebook: “Episode 04 – Probability Entropy Perplexity” (Colab), linked in the description. Sources on every slide.

PART 1 · PROBABILITY

Numbers that add to one

Each outcome gets a number from 0 to 1, and together they sum to exactly 1.

01 0.30 + 0.20 + 0.15 + 0.35 = 1.00

02 Conditional: given “The cat sat on the”



Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 3 §§3.6, 3.13. Episode 1’s probabilities (mat 0.30, floor 0.20, sofa 0.15, other 0.35) are illustrative, not measured.

PART 1 · THE CHAIN RULE

A sentence is a product

$$P(x_1, \ldots, x_n) = \prod p(x_t \mid x_{<t})$$

01 $p(x_1) \cdot p(x_2 \mid x_1) \cdot p(x_3 \mid x_1, x_2) \cdot \ldots$

02 Each factor conditions on everything before it

FIG. 5 CHAIN RULE

P = product

chain rule of probability

$$P(x_1, \ldots, x_n) = \prod_{t=1..n} p(x_t \mid x_{<t})$$



P(sentence) =

p(The)	$p(x_1)$
× p(cat | The)	$p(x_2 \mid x_1)$
× p(sat | The cat)	$p(x_3 \mid x_1, x_2)$
× p(on | The cat sat)	$p(x_4 \mid x_{<4})$
× … to the end	

Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 3 §§3.6, 3.13. Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft, ch. 3 (perplexity).

PART 1 · BY HAND

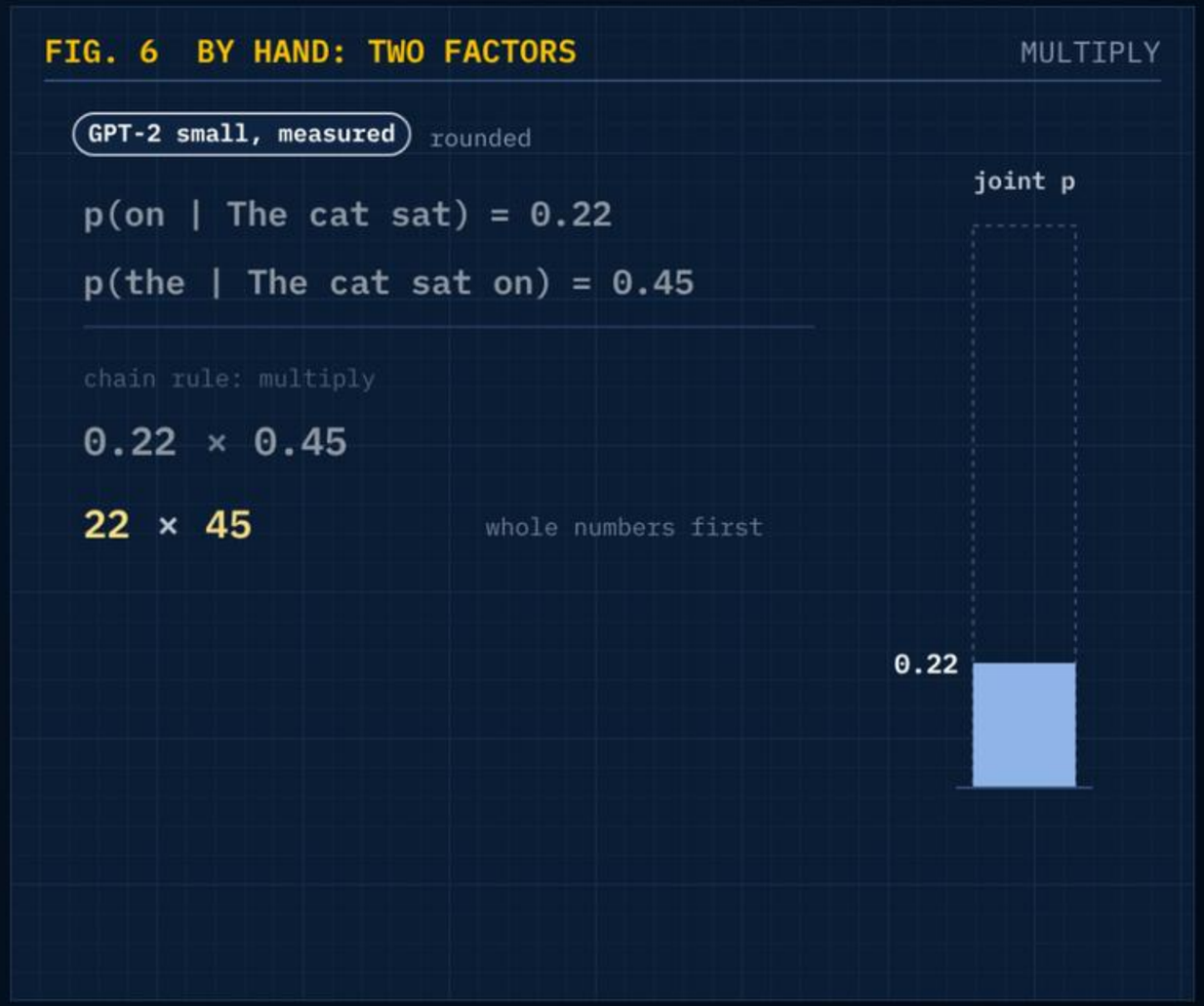
Multiply two real factors

GPT-2 small, from the Episode 1 notebook (rounded).

01 p(on | The cat sat) = 0.22

02 p(the | The cat sat on) = 0.45

03 0.22 × 0.45 = 0.099: under 10% after two words



GPT-2 small probabilities measured in the Episode 1 companion notebook, rounded.

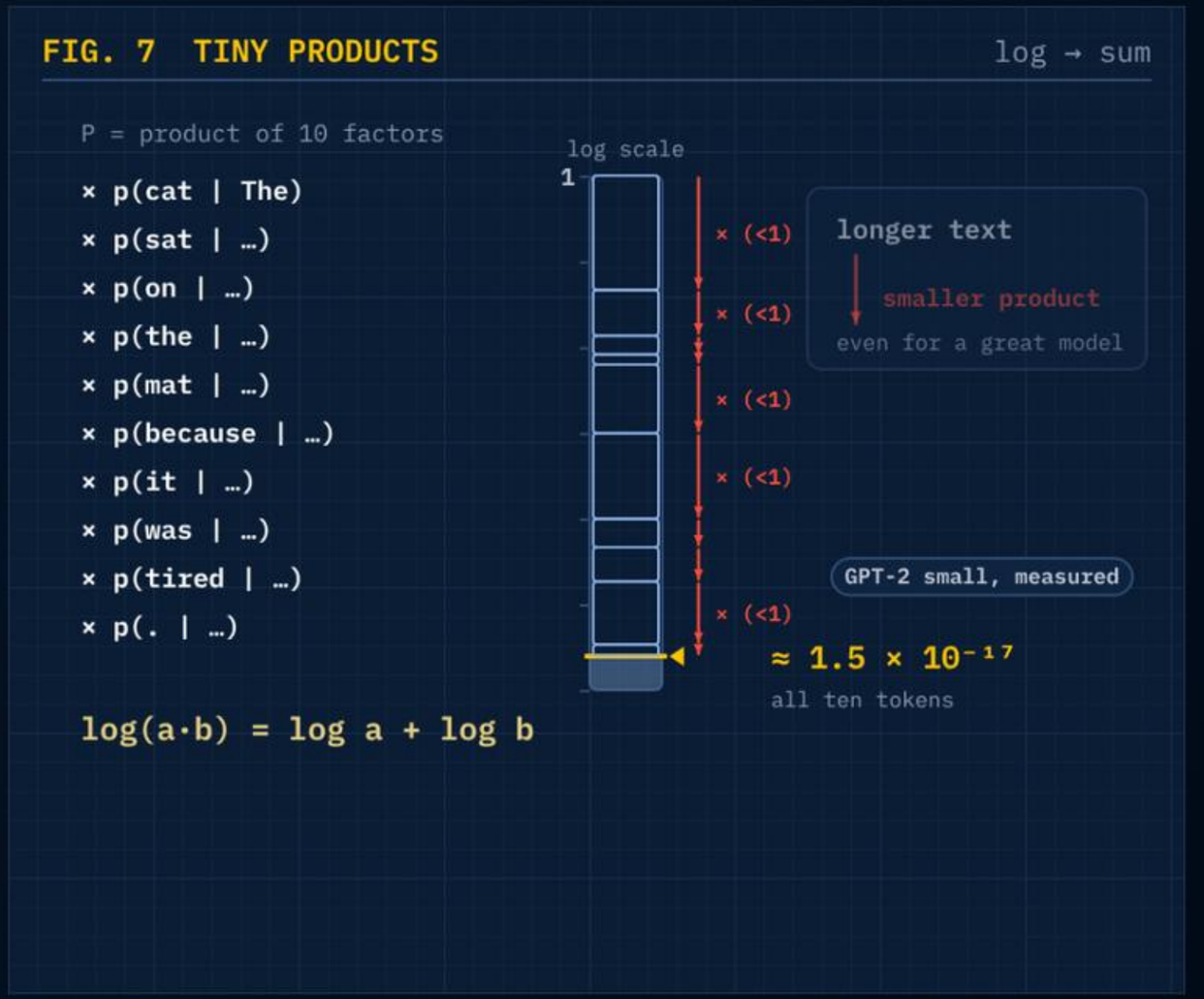
Products shrink fast

All ten scored tokens of the sentence: about 1.5 × 10^{-17}.

01 Every factor is below 1, so every product shrinks

02 Longer text, smaller number, even for a great model

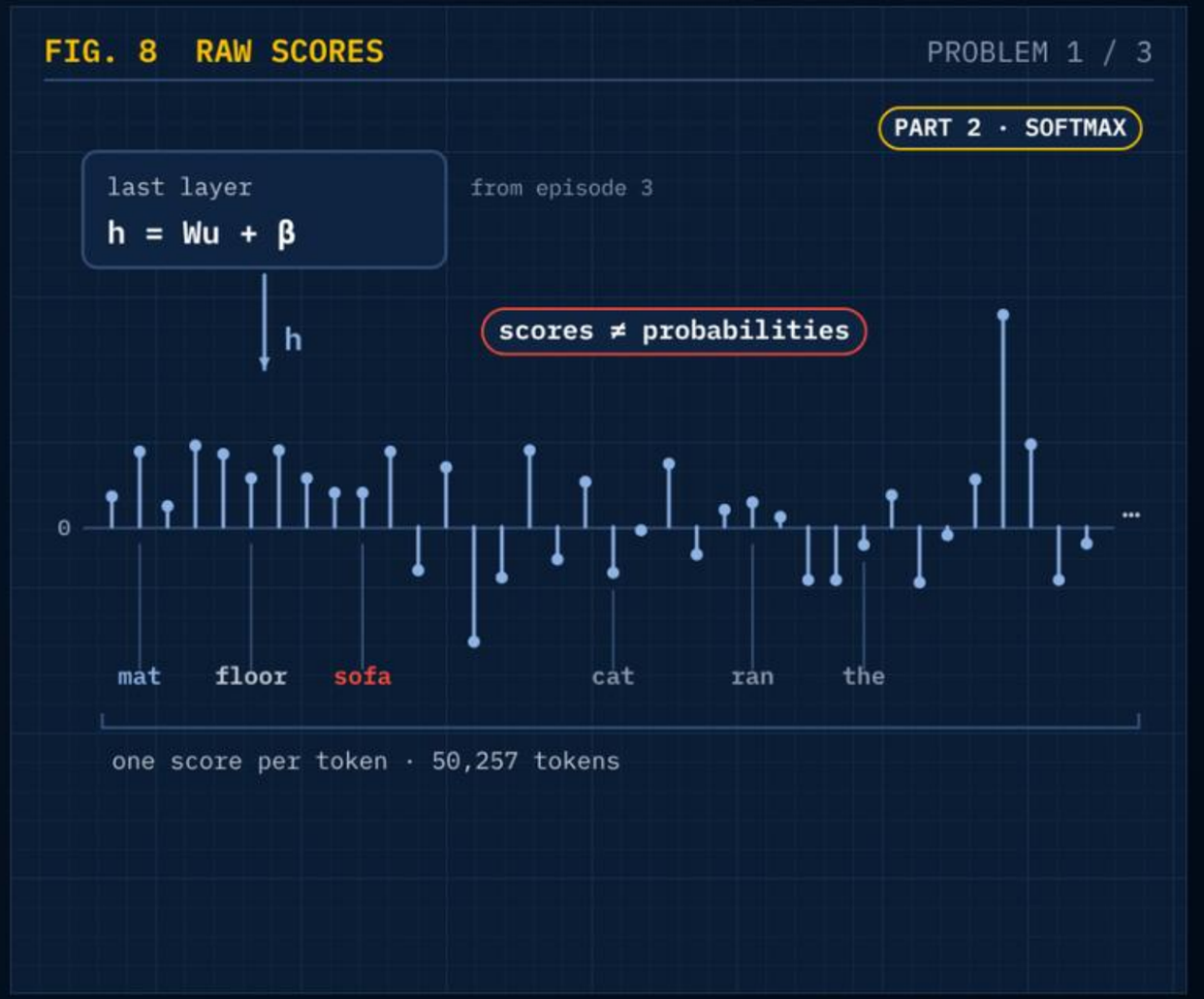
03 log(a·b) = log a + log b: products become sums



GPT-2 small probabilities measured in the Episode 1 companion notebook, rounded.

A score for every token

The last layer outputs h: one number per token in the vocabulary.



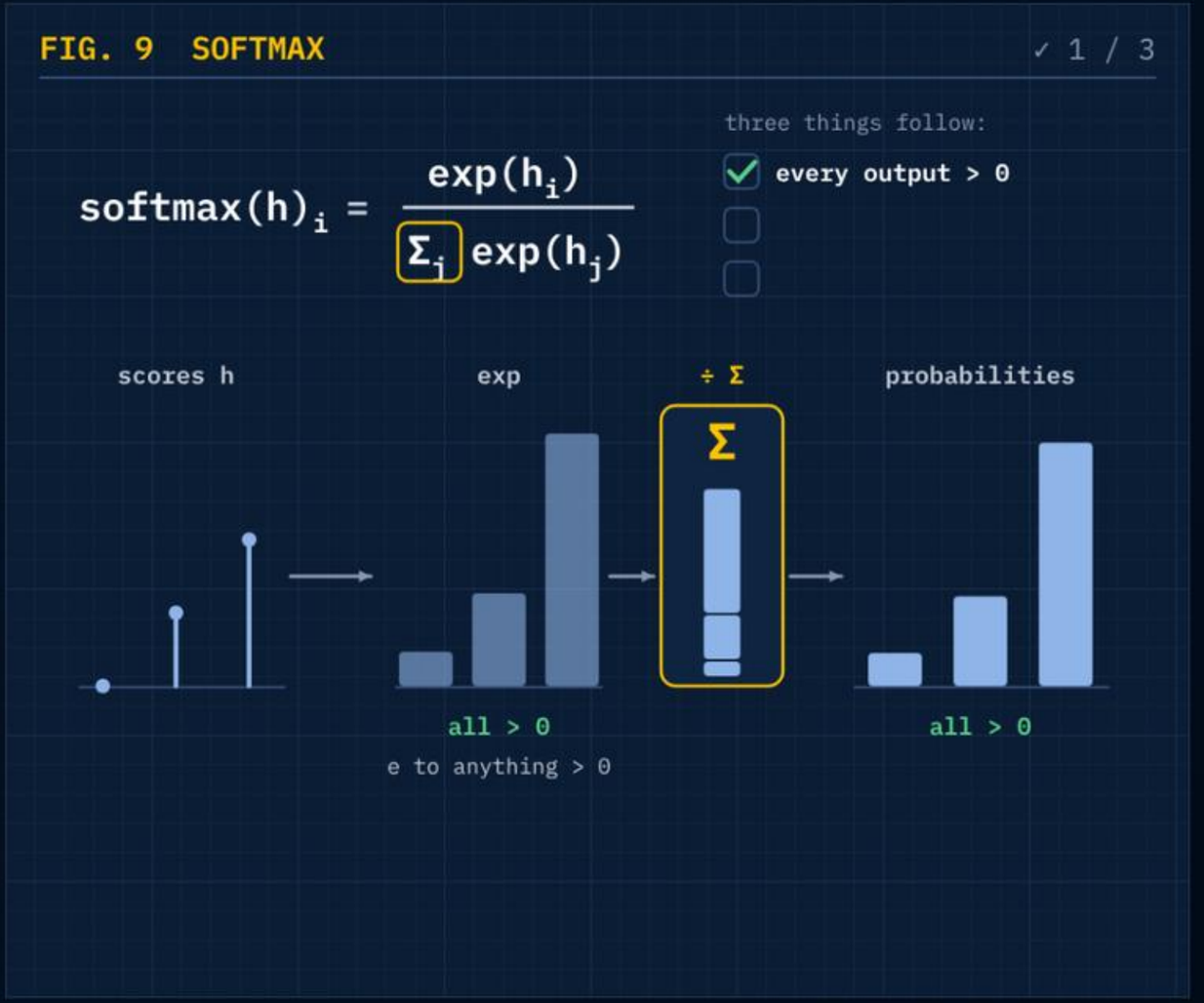
h is the layer output from Episode 3. Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 6 §6.2.2.3 (softmax, stable form).

PART 2 · SOFTMAX

Exponentiate, then divide

$$\text{softmax}(h)_i = \exp(h_i) \,/\, \Sigma_j \exp(h_j)$$

01 Every output is positive: exp is positive



Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 6 §6.2.2.3 (softmax, stable form).

PART 2 · BY HAND, STEPS 1–2

Exponentiate, then add

h = (0, 1, 2)

01 $e^0 = 1 \cdot e^1 = 2.718 \cdot e^2 = 7.389$

02 $1 + 2.718 = 3.718$

FIG. 10 SOFTMAX BY HAND — STEP 2 / 3

three scores, by hand

$h = (0, 1, 2)$

STEP 1 exponentiate each

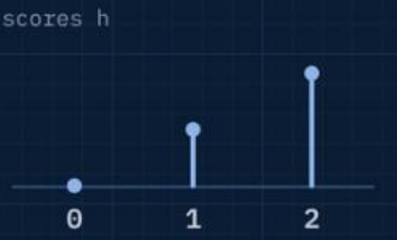
$e^0 = 1$

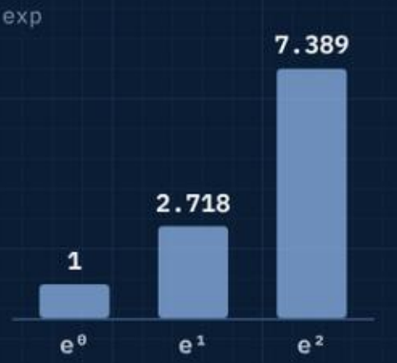
$e^1 = 2.718$

$e^2 = 7.389$

STEP 2 add them up

$1 + 2.718 = 3.718$





Σ running total

3.718

Worked example; every number is checked in the companion notebook.

PART 2 · BY HAND, STEP 3

Divide by the total

The same denominator, 11.107, for every entry.

01 1 ÷ 11.107 = 0.090

02 2.718 ÷ 11.107 = 0.245

03 7.389 ÷ 11.107 = 0.665

04 Sum: 1.000. Each +1 in score: × 2.72



Worked example; every number is checked in the companion notebook.

PART 2 · THE SHIFT CHECK

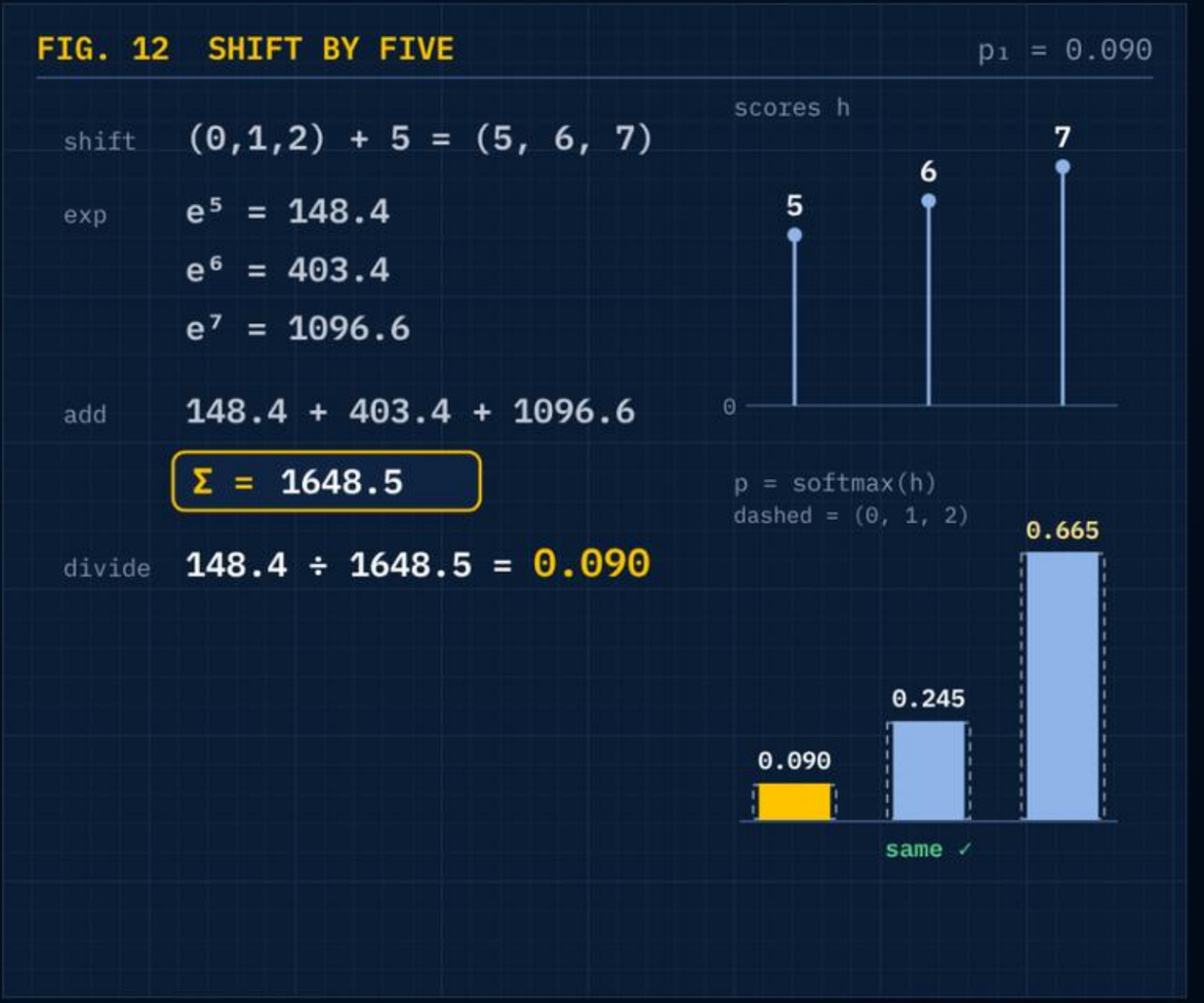
Add 5 to every score

$h + 5 = (5, 6, 7)$

01 $e^5 = 148.4 \cdot e^6 = 403.4 \cdot e^7 = 1096.6$

02 Sum = 1648.5

03 $148.4 \div 1648.5 = 0.090$: the same answer



Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 6 §6.2.2.3 (softmax, stable form). Worked example; every number is checked in the companion notebook.

PART 2 · HOW REAL CODE DOES IT

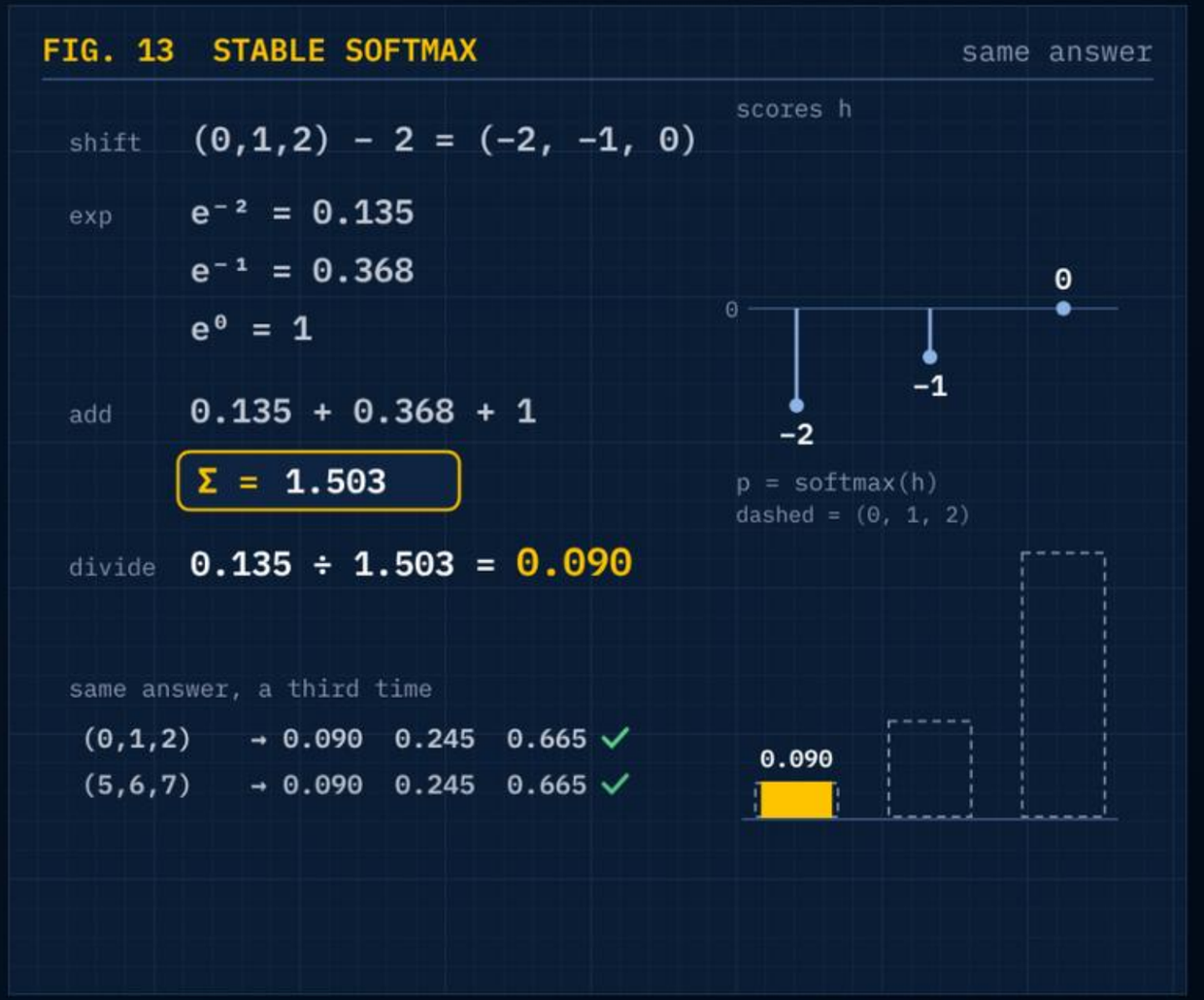
Subtract the largest score

$h - 2 = (-2, -1, 0)$

01 $e^{-2} = 0.135 \cdot e^{-1} = 0.368 \cdot e^{0} = 1$

02 Sum = 1.503

03 $0.135 \div 1.503 = 0.090$: same, a third time



Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 6 §6.2.2.3, Eq. 6.33 (numerically stable form).

PART 2 · BACK TO EPISODE 1

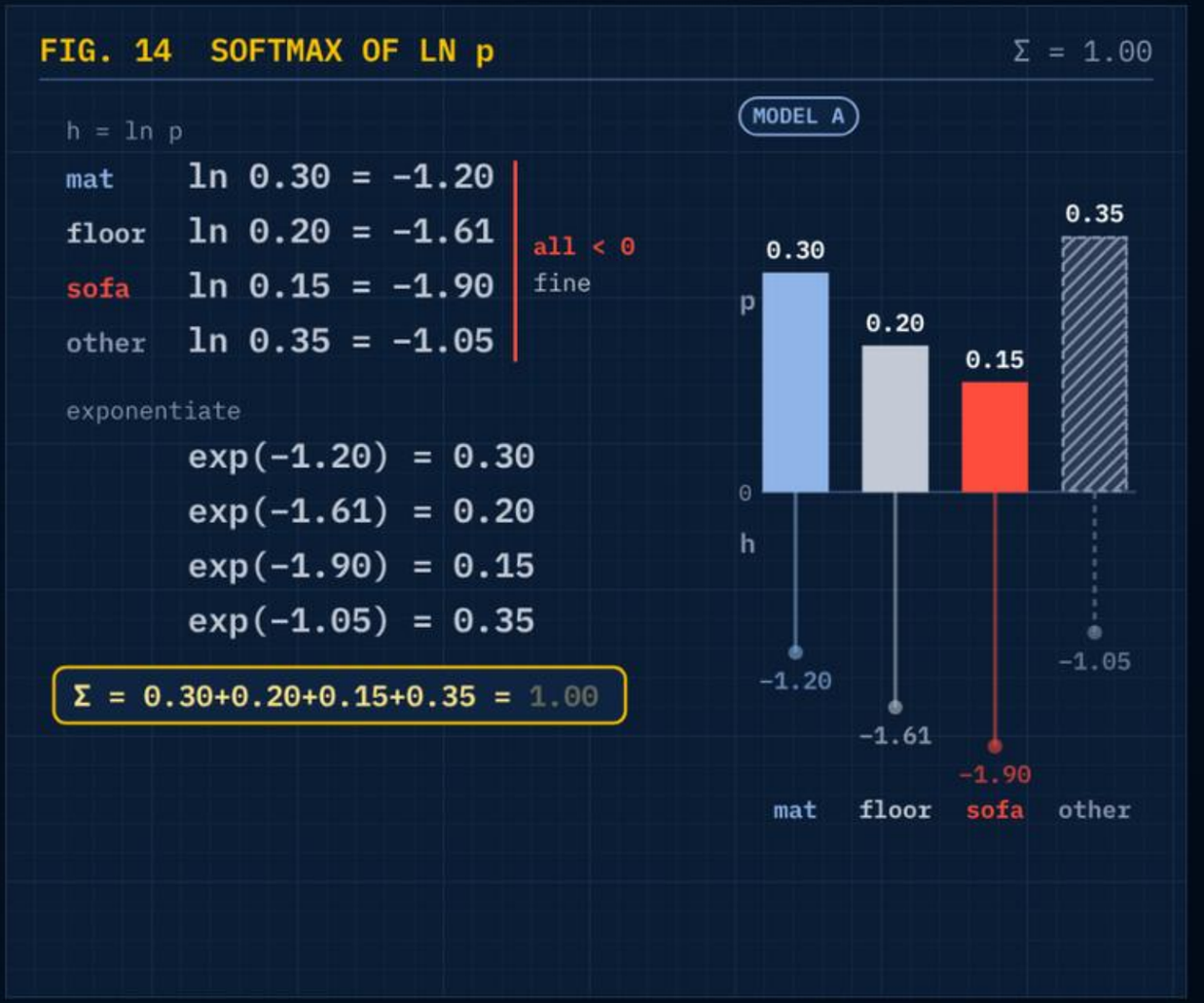
Softmax of ln p gives p

$$h = \ln p \approx (-1.20, -1.61, -1.90, -1.05)$$

01 All negative, and that's fine

02 exp gives back 0.30, 0.20, 0.15, 0.35

03 They already sum to 1, so softmax(h) = p



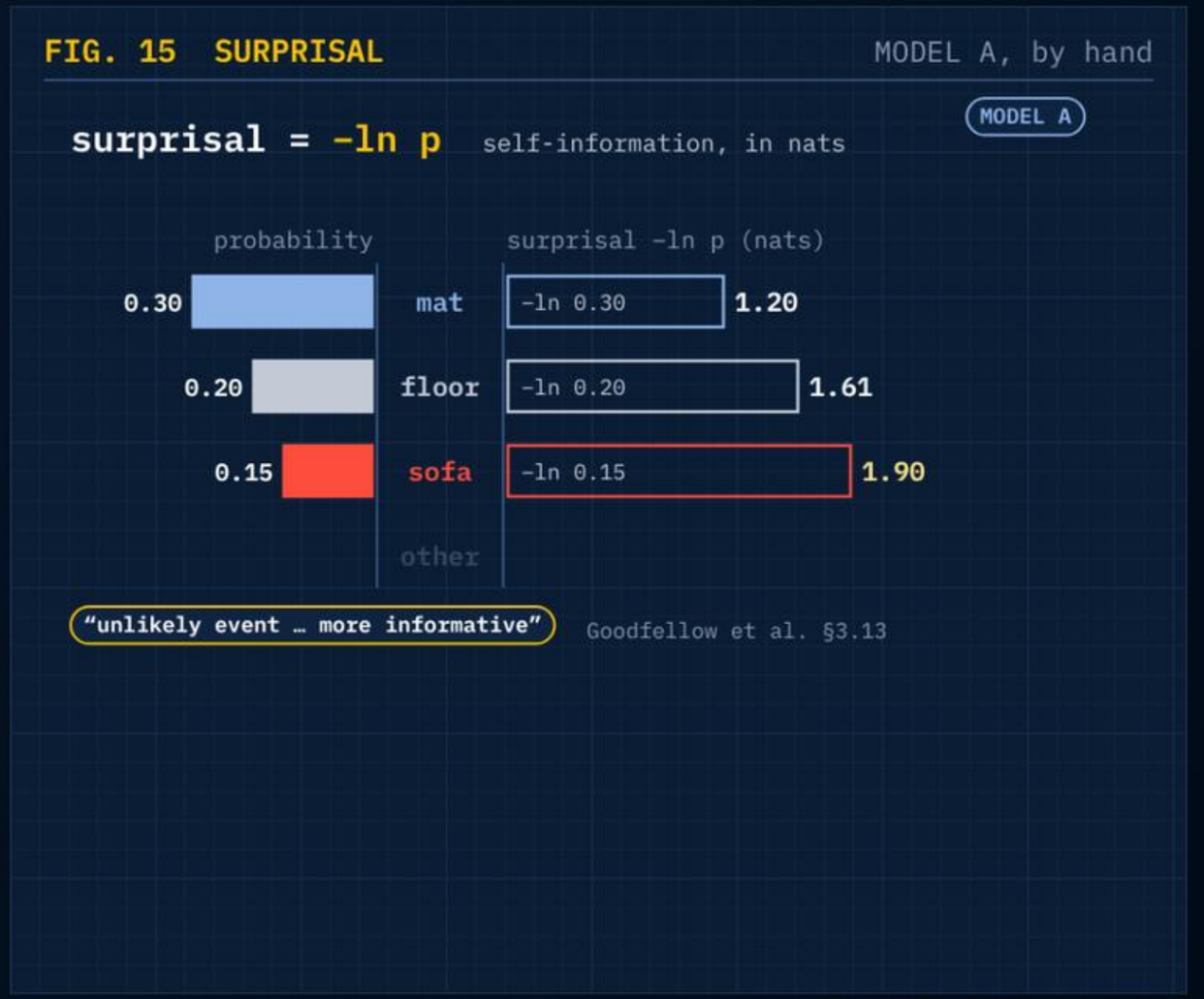
Episode 1's probabilities (mat 0.30, floor 0.20, sofa 0.15, other 0.35) are illustrative, not measured. Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 6 §6.2.2.3 (softmax, stable form).

PART 3 · SURPRISAL

Rare means surprising

Surprisal = −ln p. Goodfellow et al. call it self-information, in nats.

01 mat 1.20 · floor 1.61 · sofa 1.90 · other 1.05



Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 3 §§3.6, 3.13. Episode 1's probabilities (mat 0.30, floor 0.20, sofa 0.15, other 0.35) are illustrative, not measured.

Same surprise, new unit

bits = nats ÷ ln 2 = nats ÷ 0.693

01 mat: 1.20 ÷ 0.693 = 1.74 bits

02 floor 2.32 · sofa 2.74 · other 1.51 bits

FIG. 16 NATS TO BITS

mat: 1.74 bits

word	nats	÷ ln 2	bits
mat	1.20	÷ 0.693 =	1.74
floor	1.61	÷ 0.693 =	2.32
sofa	1.90	÷ 0.693 =	
other	1.05		

ln → nats
$\log_2$ → bits

bits = nats ÷ ln 2

ln 2 = 0.693

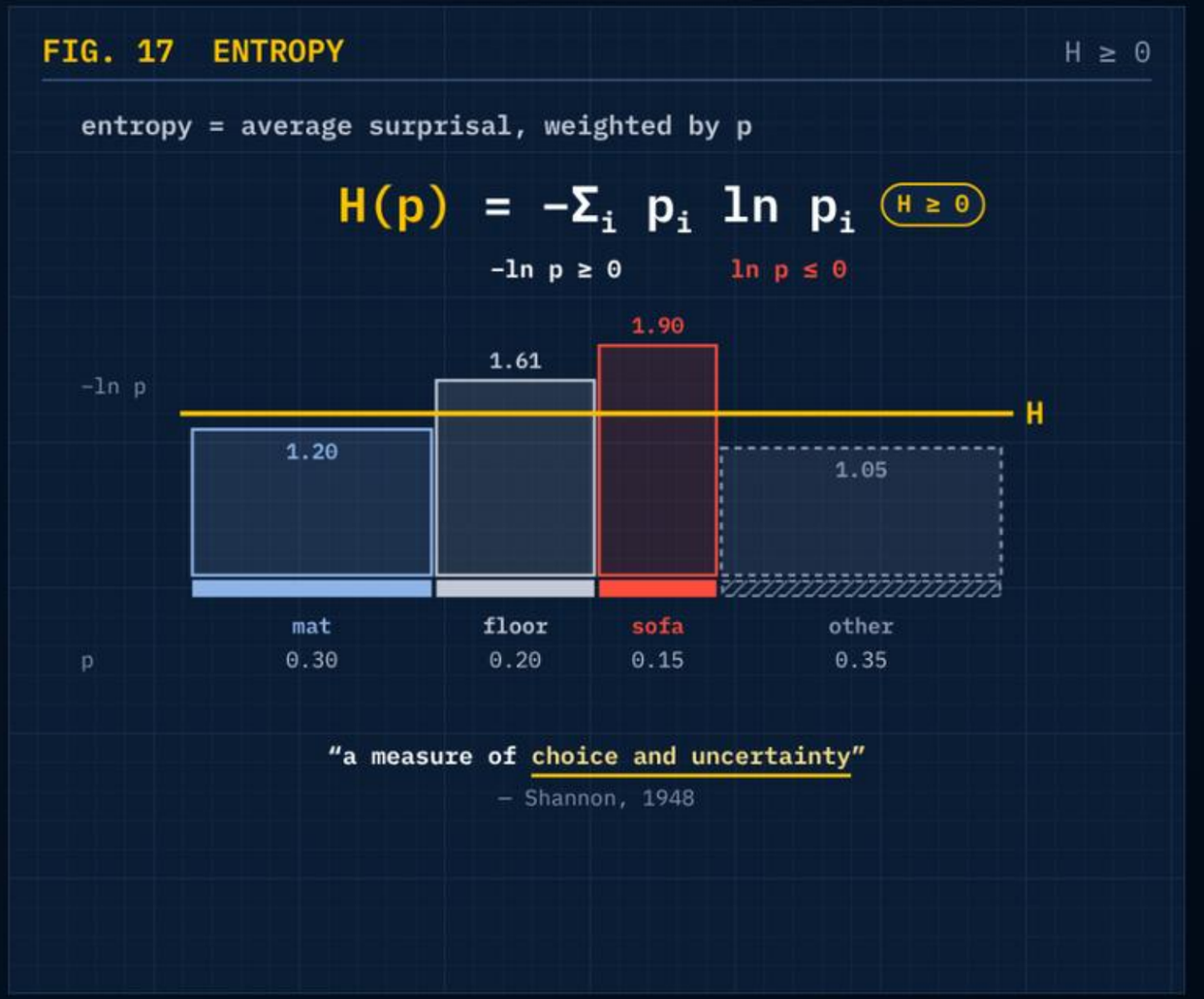
Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal* 27 (1948). Base 2 gives bits.

Average surprise

$$H(p) = -\Sigma_i \; p_i \; \ln \; p_i$$

01 Each surprisal weighted by how often it happens

02 The minus sign makes H zero or more



Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal* 27 (1948). Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 3 §§3.6, 3.13.

PART 3 · BY HAND, EXAMPLE 1

Four equal outcomes

$p = (0.25, 0.25, 0.25, 0.25)$

01 $-\ln 0.25 = 1.386$

02 Each term: $0.25 \times 1.386 = 0.347$

03 $4 \times 0.347 = 1.386$ nats $= 2$ bits

FIG. 18 BY HAND: FOUR EQUAL — H = 2 bits

four outcomes, equally likely, 0.25 each

term = 0.25 × (−ln 0.25)

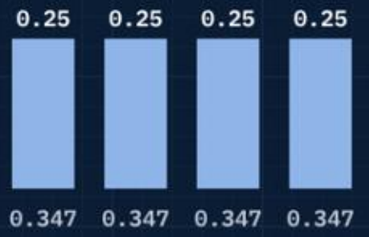
−ln 0.25 = 1.386

term = 0.25 × 1.386 = 0.347

0.347 + 0.347 + 0.347 + 0.347

= **1.386 nats**

1.386 nats ÷ 0.693 = 2 bits



Worked example; every number is checked in the companion notebook.

PART 3 · BY HAND, EXAMPLE 2

Episode 1's model

One term per word: probability × surprisal.

01 0.30 × 1.204 = 0.361 · 0.20 × 1.609 = 0.322

02 0.15 × 1.897 = 0.285 · 0.35 × 1.050 = 0.367

03 H = 1.335 nats = 1.93 bits

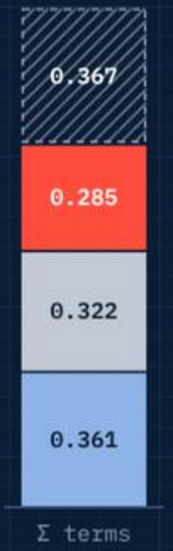
FIG. 19 BY HAND: MODEL A

ADD

MODEL A one term per word: p × surprisal

word	p	−ln p	term
mat	0.30	1.204	0.361
floor	0.20	1.609	0.322
sofa	0.15	1.897	0.285
other	0.35	1.050	0.367

H = 0.361 + 0.322 + 0.285 + 0.367



Episode 1's probabilities (mat 0.30, floor 0.20, sofa 0.15, other 0.35) are illustrative, not measured. Worked example; every number is checked in the companion notebook.

PART 3 · BY HAND, EXAMPLE 3

The confident model

sofa 0.97 · mat, floor, other 0.01 each

01 0.97 × 0.030 ≈ 0.030

02 0.01 × 4.605 = 0.046, three times: 0.138

03 H ≈ 0.17 nats, against 1.34: nearly 8× more sure

FIG. 20 BY HAND: MODEL B

A vs B

MODEL B

sofa: p = 0.97

−ln 0.97 = 0.030

term = 0.97 × 0.030 ≈ 0.030

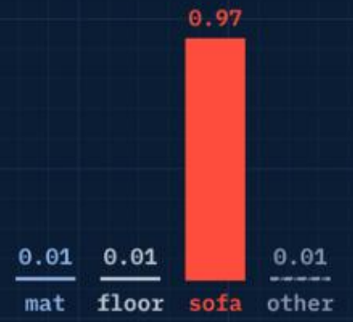
mat, floor, other: p = 0.01

−ln 0.01 = 4.605

term = 0.01 × 4.605 = 0.046

0.046 × 3 = 0.138

H ≈ 0.030 + 0.138 ≈ 0.17 nats



the cold open's answer

Worked example; every number is checked in the companion notebook.

“Other” is a bucket

Splitting a lumped outcome can only keep H the same or raise it.

01 1.335 nats is a lower bound for that model

02 GPT-2 small at the same point: about 5.4 nats

03 That’s over all 50,257 tokens



Grouping bound derived from the definition of H. GPT-2 small probabilities measured in the Episode 1 companion notebook, rounded.

A PICTURE TO KEEP

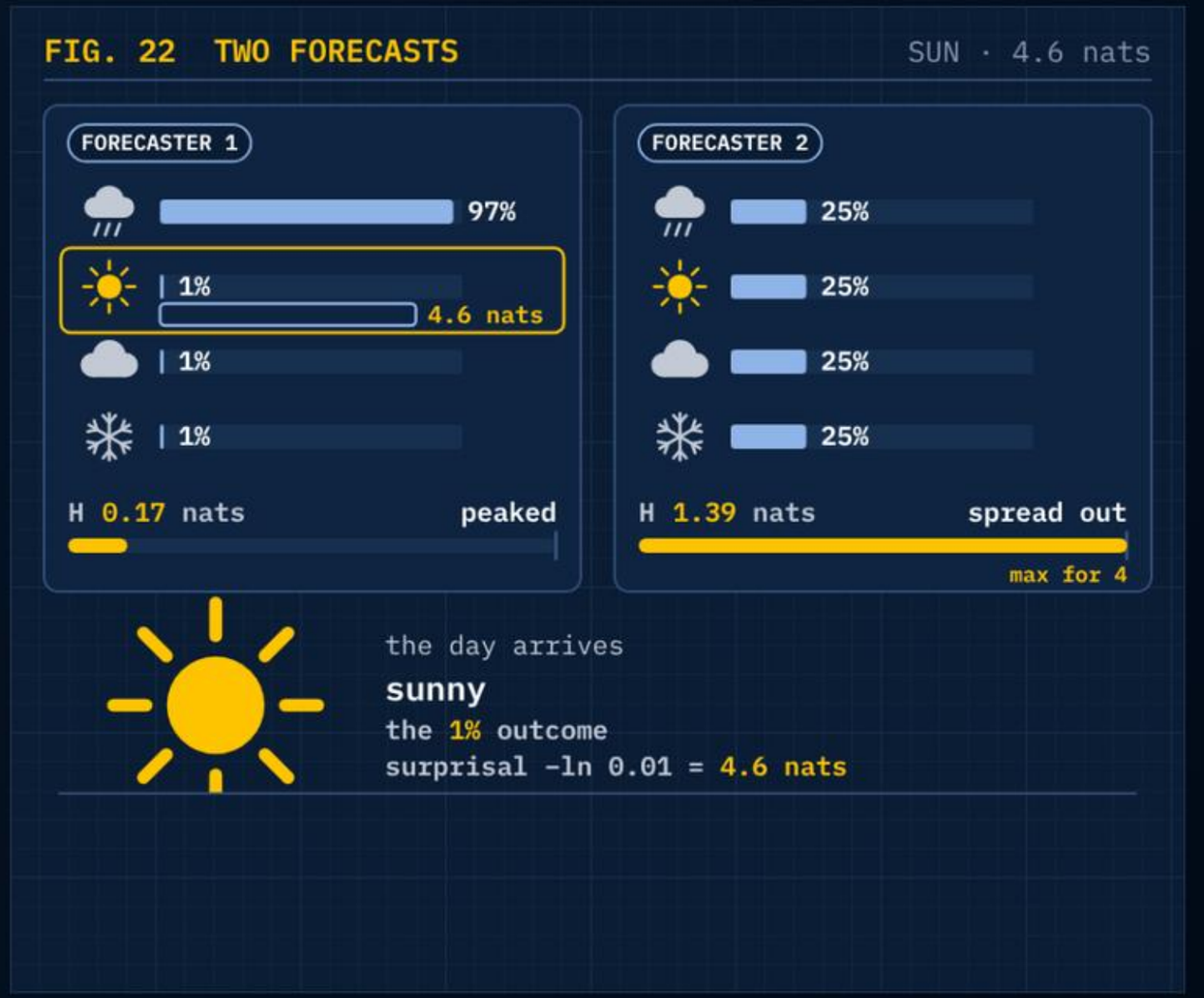
The weather forecast

Spread out = unsure. Peaked = confident, not always right.

01 97% rain: about 0.17 nats

02 Four kinds of weather, a quarter each: 1.39 nats

03 Then the 1% day arrives: surprisal 4.6 nats



Kinvecta analogy, built for this series; numbers match the 0.97 example.

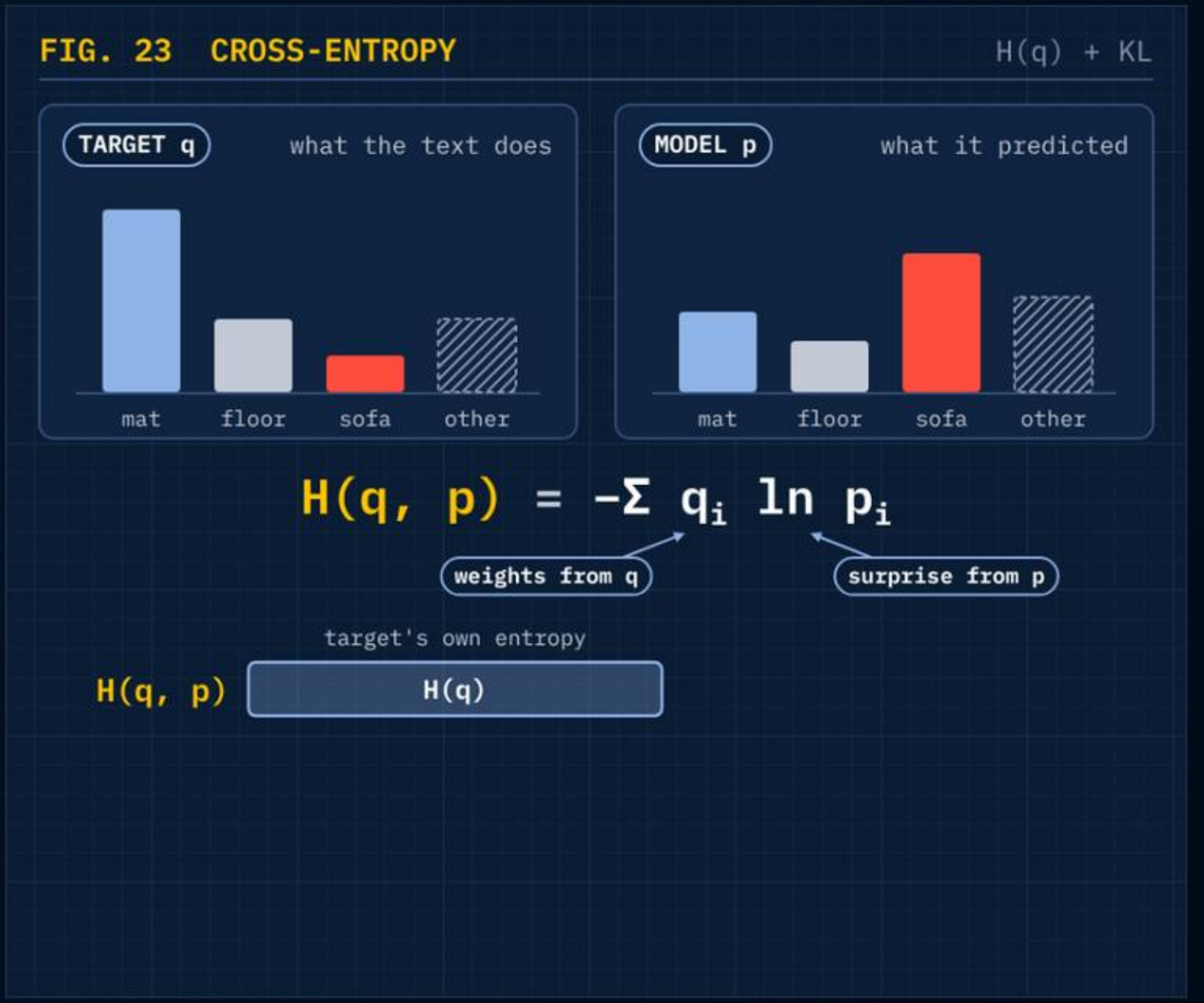
PART 4 · CROSS-ENTROPY

Model versus target

$$H(q, p) = -\Sigma_i \, q_i \, \ln p_i$$

01 q: what the text does · p: what the model predicted

02 H(q, p) = H(q) + D_KL(q ‖ p)



Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 3 §§3.6, 3.13. (Eqs. 3.50–3.51).

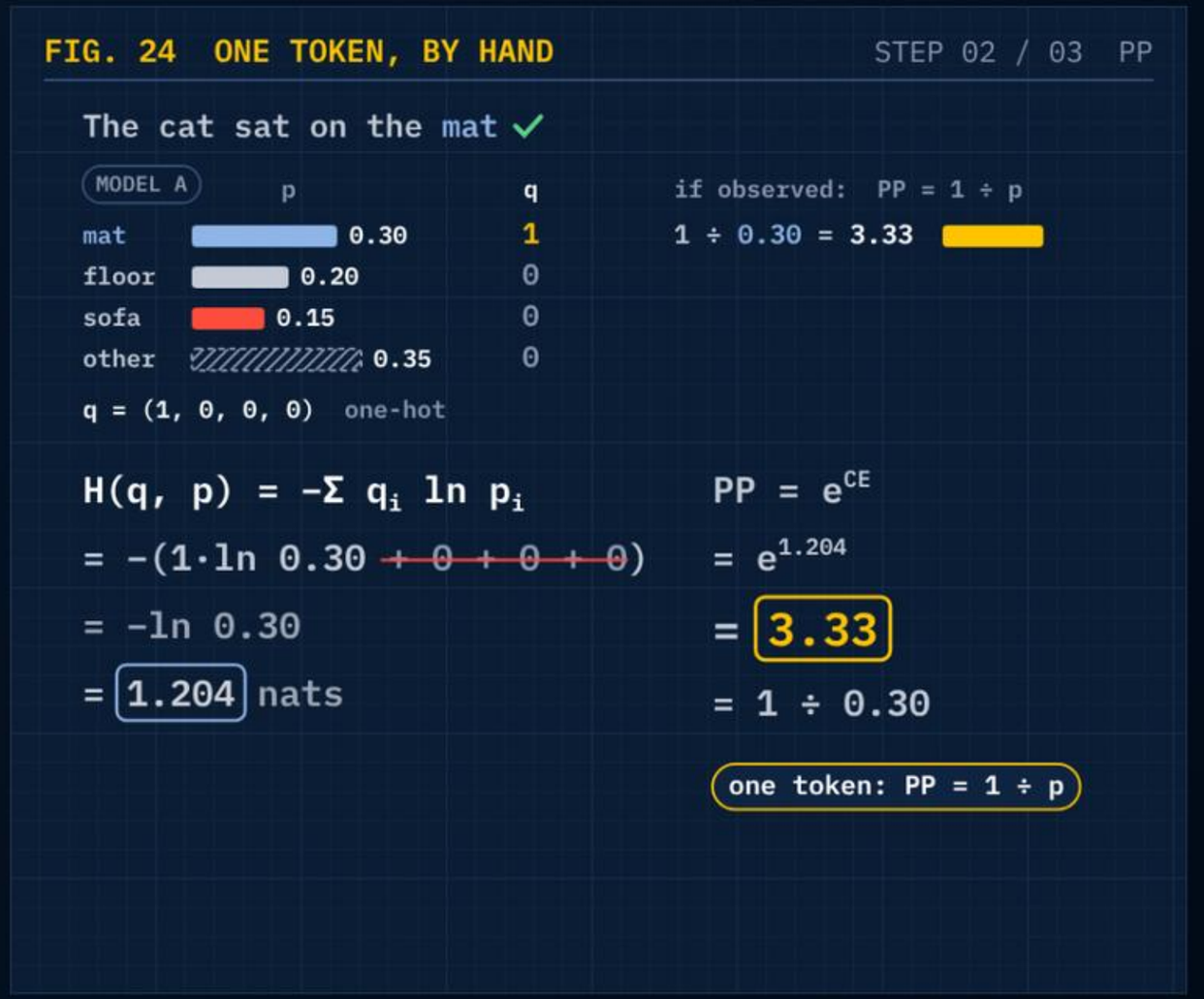
PART 4 · BY HAND, ONE TOKEN

The next word is mat

q puts all its weight on mat, so one term is left.

01 −ln 0.30 = 1.204 nats

02 PP = e^1.204 = 3.33 = 1 ÷ 0.30



Episode 1's probabilities (mat 0.30, floor 0.20, sofa 0.15, other 0.35) are illustrative, not measured. Worked example; every number is checked in the companion notebook.

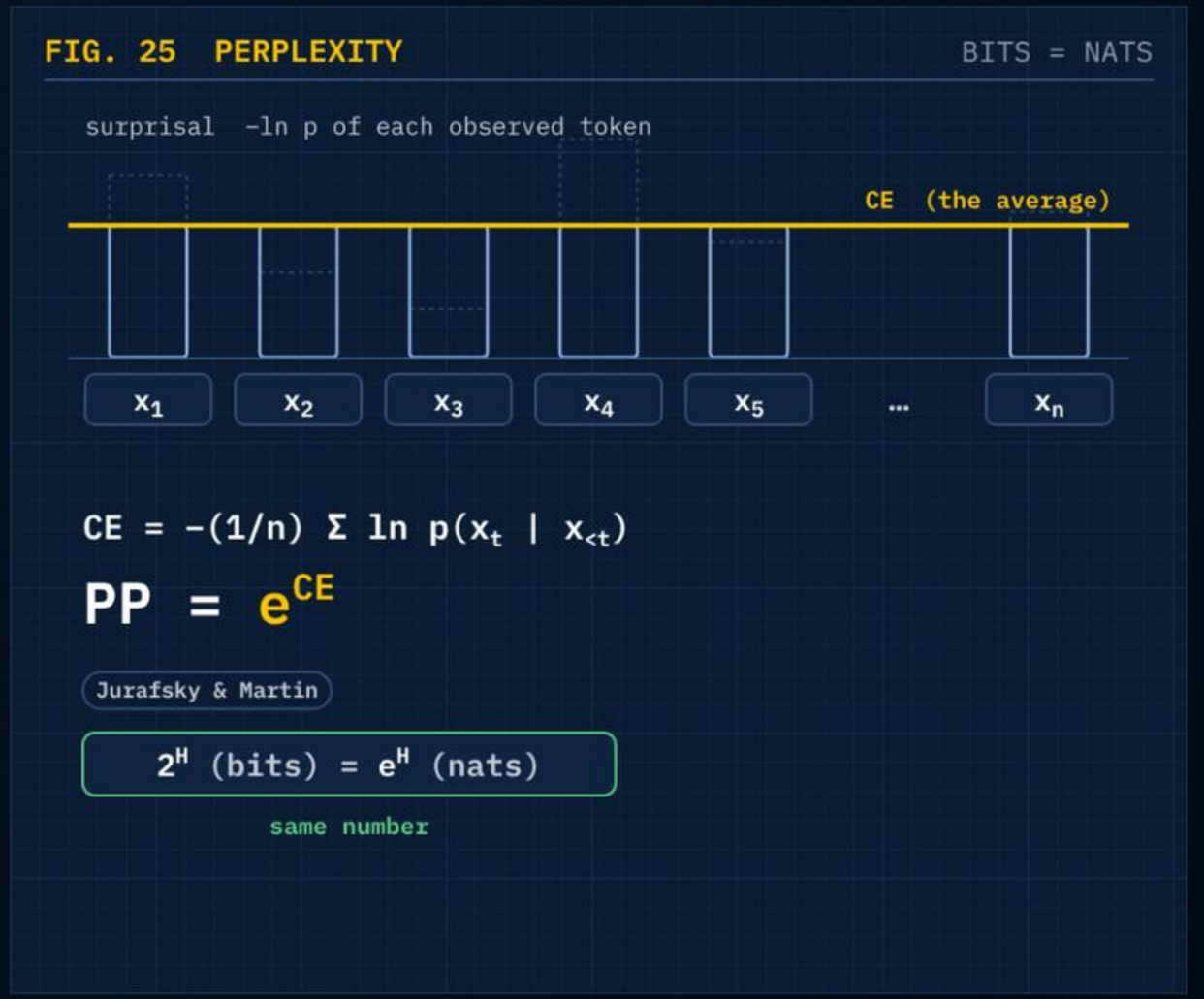
PART 4 · PERPLEXITY

e to the average surprise

$$CE = -(1/n)\ \Sigma_t\ \ln\ p(x_t \mid x_{<t})\ \cdot\ PP = \exp(CE)$$

01 Average over the tokens that actually appear

02 Jurafsky & Martin: PP = 2^H in bits, the same number



Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft, ch. 3 (perplexity).

PART 4 · BY HAND, TWO TOKENS

GPT-2 on “on the”

`-ln 0.22 = 1.51 · -ln 0.45 = 0.80`

01 1.51 + 0.80 = 2.31; ÷ 2 = 1.16 nats per token

02 PP = e^1.16 ≈ 3.2

03 Check: 1 ÷ √0.099 ≈ 3.2, one over the geometric mean

FIG. 26 PERPLEXITY BY HAND — ROUTE 2 / 2

GPT-2 small, measured

p(on | The cat sat) = 0.22
p(the | The cat sat on) = 0.45

ROUTE 1 · average surprisal

-ln 0.22 = 1.51
-ln 0.45 = 0.80
1.51 + 0.80 = 2.31
2.31 ÷ 2 = 1.16 nats/token
PP = $e^{1.16}$ ≈ 3.2

ROUTE 2 · the product

0.22 × 0.45 =

GPT-2 small probabilities measured in the Episode 1 companion notebook, rounded.

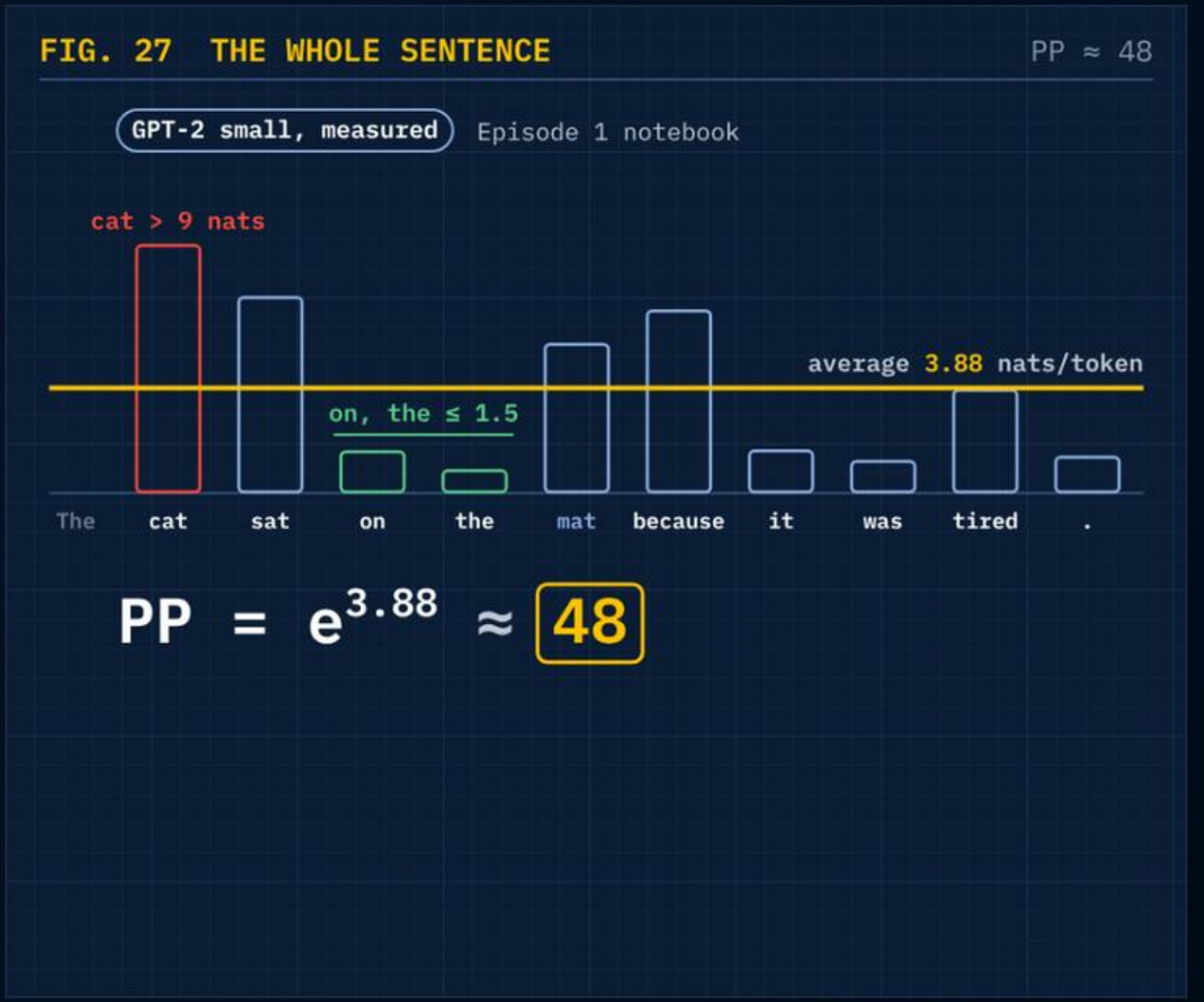
PART 4 · THE WHOLE SENTENCE

GPT-2 small: perplexity 48

Ten scored tokens, average surprisal 3.88 nats.

01 PP = e^3.88 ≈ 48

02 Easy: “on”, “the” ≤ 1.5 nats · hard: “cat” > 9



GPT-2 small probabilities measured in the Episode 1 companion notebook, rounded. Episode 1's probabilities (mat 0.30, floor 0.20, sofa 0.15, other 0.35) are illustrative, not measured.

MYTH CHECK

“Perplexity is the number of choices”

Exact only for a uniform distribution.

01 Four equal outcomes: PP = 4

02 Otherwise: the number of equal choices with the same H

03 On real text, PP ≥ e^H(q), equal only when the model matches



Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft, ch. 3 (perplexity). Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016), ch. 3 §§3.6, 3.13.

Kinvecta

MYTH CHECK · THE FINE PRINT

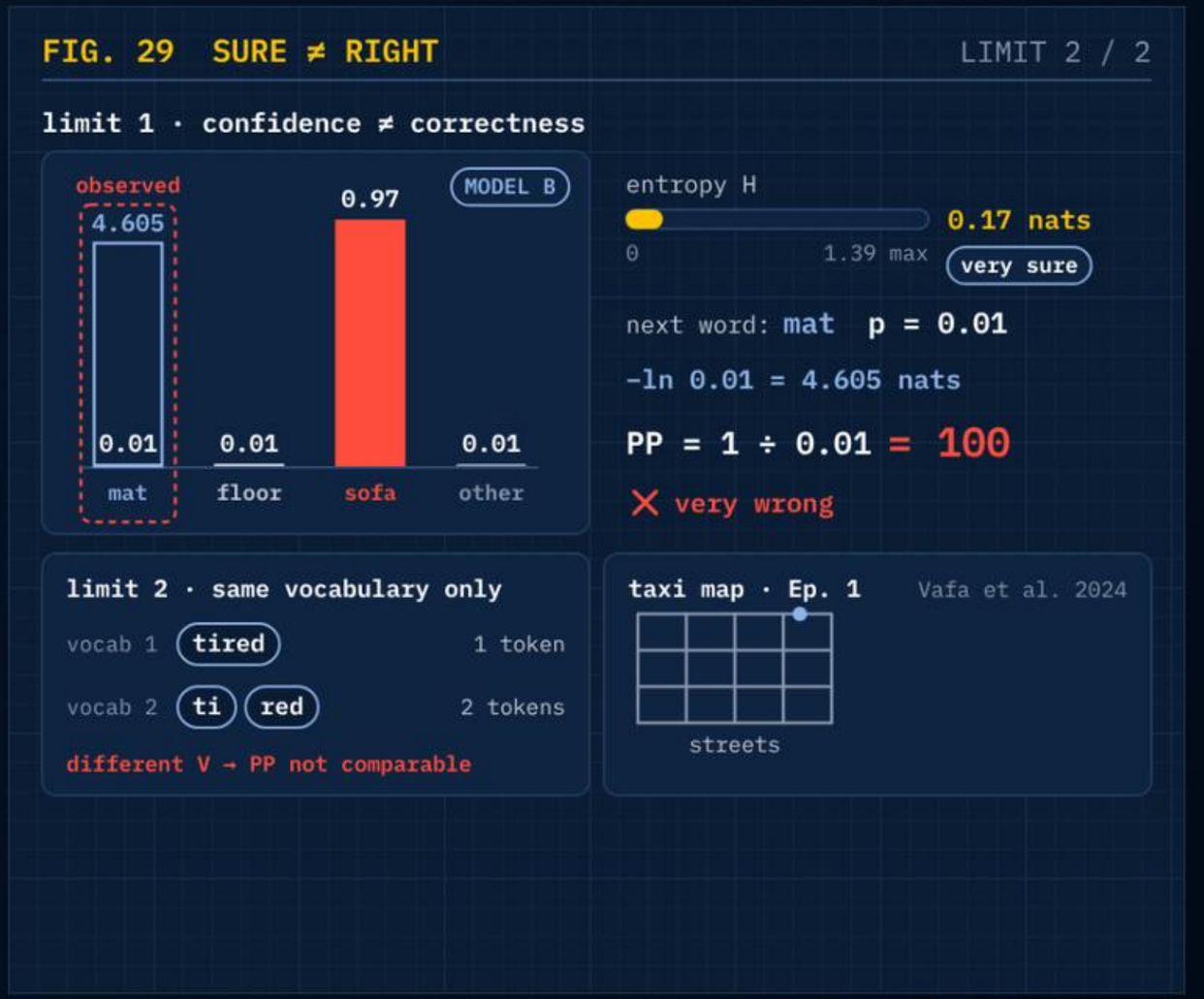
Confident is not correct

Sofa at 0.97: entropy 0.17 nats. The next word is mat, at 0.01.

01 PP on that token = 1 ÷ 0.01 = 100

02 Compare only across identical vocabularies

03 Taxi maps: near-perfect predictions, wrong streets



Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft, ch. 3 (perplexity). Vafa et al., "Evaluating the World Model Implicit in a Generative Model," NeurIPS 2024.

MARK

RECAP

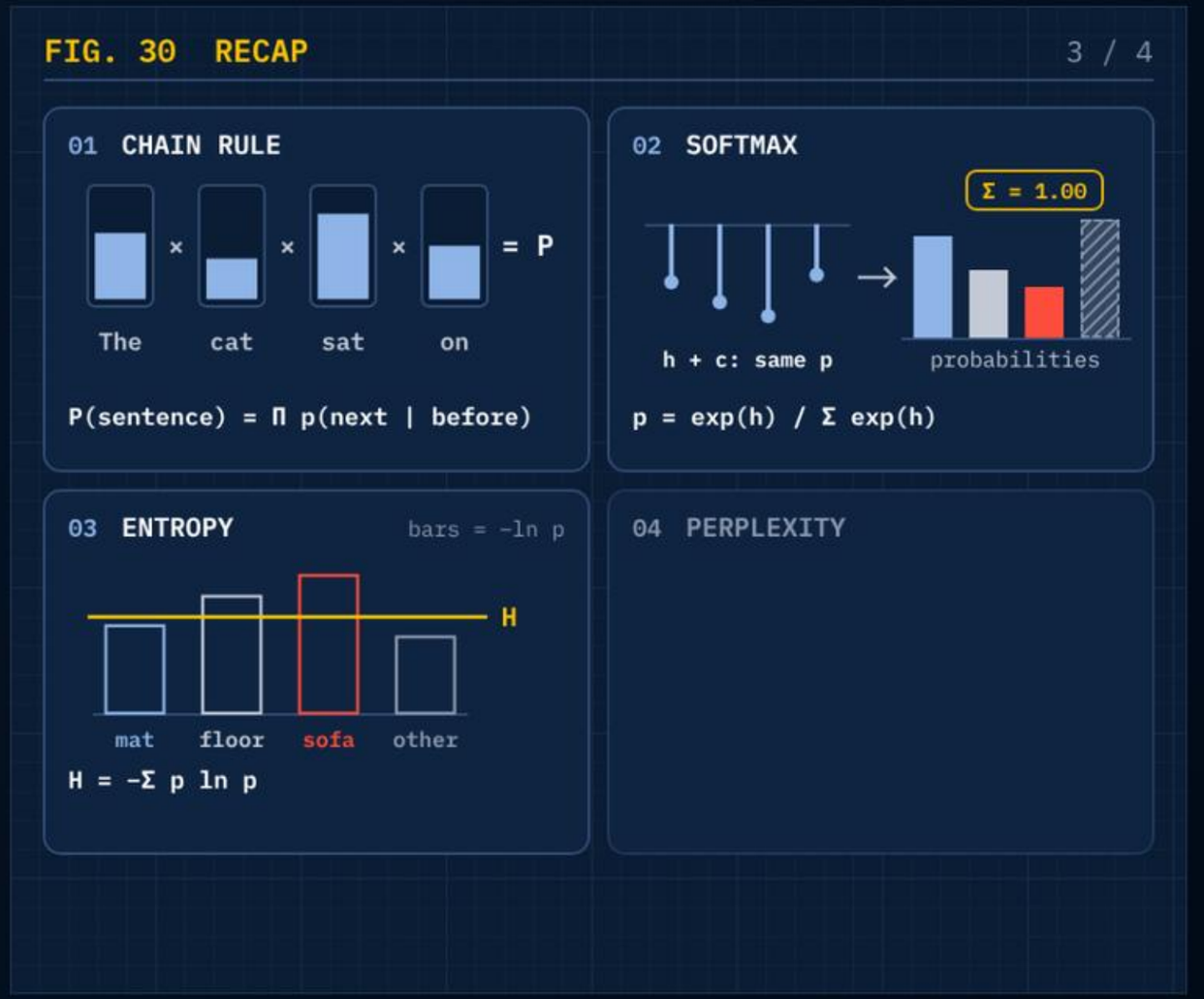
Four ideas

Every way of measuring a model starts here.

01 Chain rule: a sentence is a product of next-token probabilities

02 Softmax: exponentiate, divide; only differences matter

03 Entropy: the average surprisal of one distribution



All sources are listed in the description.

YOUR TURN

One question to take with you

A model spreads its probability evenly over four candidate words.

01 Its entropy, in nats and in bits?

02 Its perplexity?

03 mat 0.97, the rest 0.01 each: entropy higher or lower?



Worked example; every number is checked in the companion notebook.

YOUR TURN

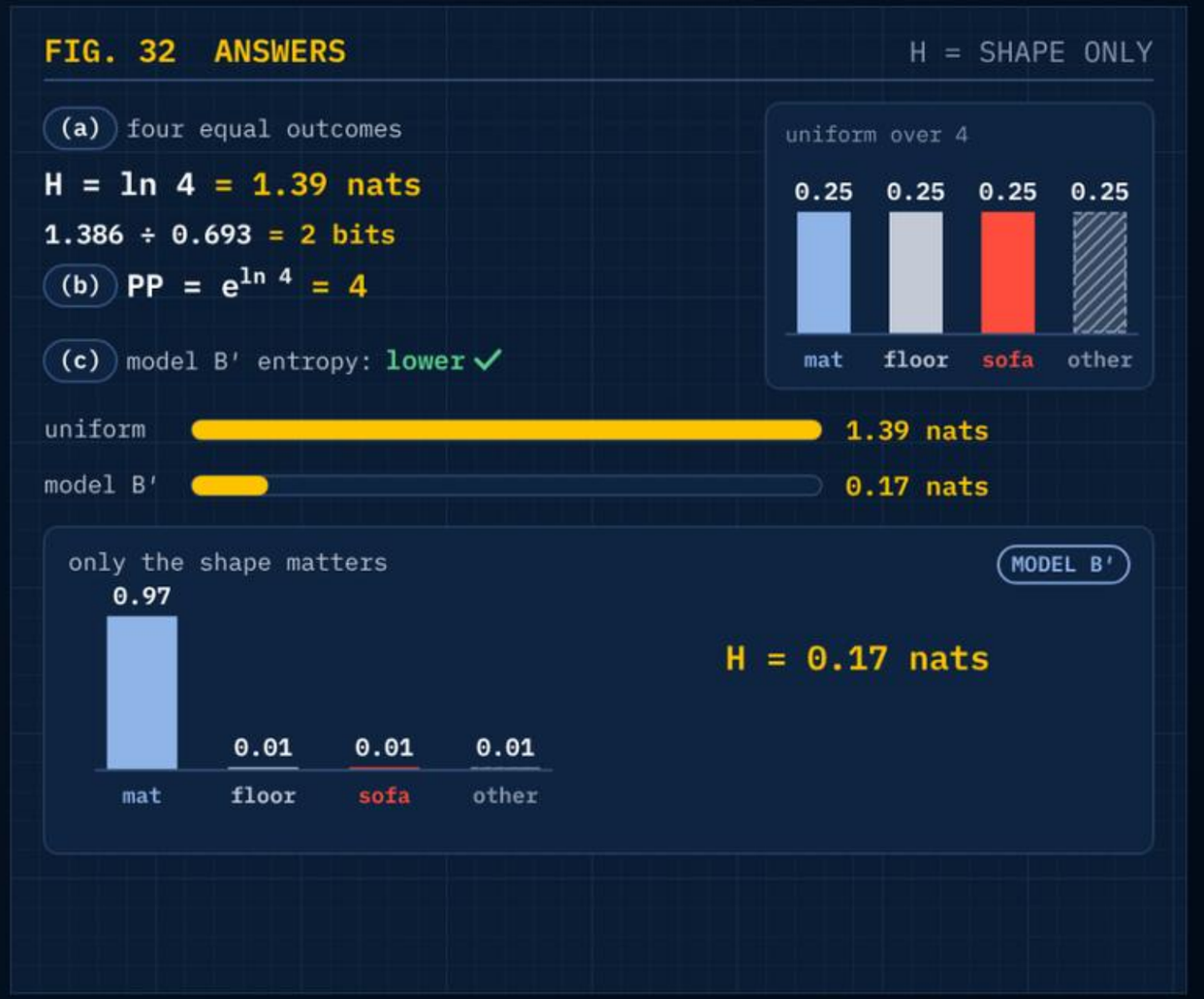
The answer

Entropy reads the shape of the distribution, not which word is on top.

01 In 4 = 1.39 nats = exactly 2 bits

02 Perplexity: 4

03 Lower: 0.17 nats against 1.39



Worked example; every number is checked in the companion notebook.

TRY THIS WEEK

Three things to try.

01

Softmax three scores of your own by hand. Then add 10 to each and redo it

02

In the notebook, even out the four bars and watch H climb to 1.39

03

Compute the sentence's perplexity of 48 from its ten probabilities

Companion notebook and every source in the description.

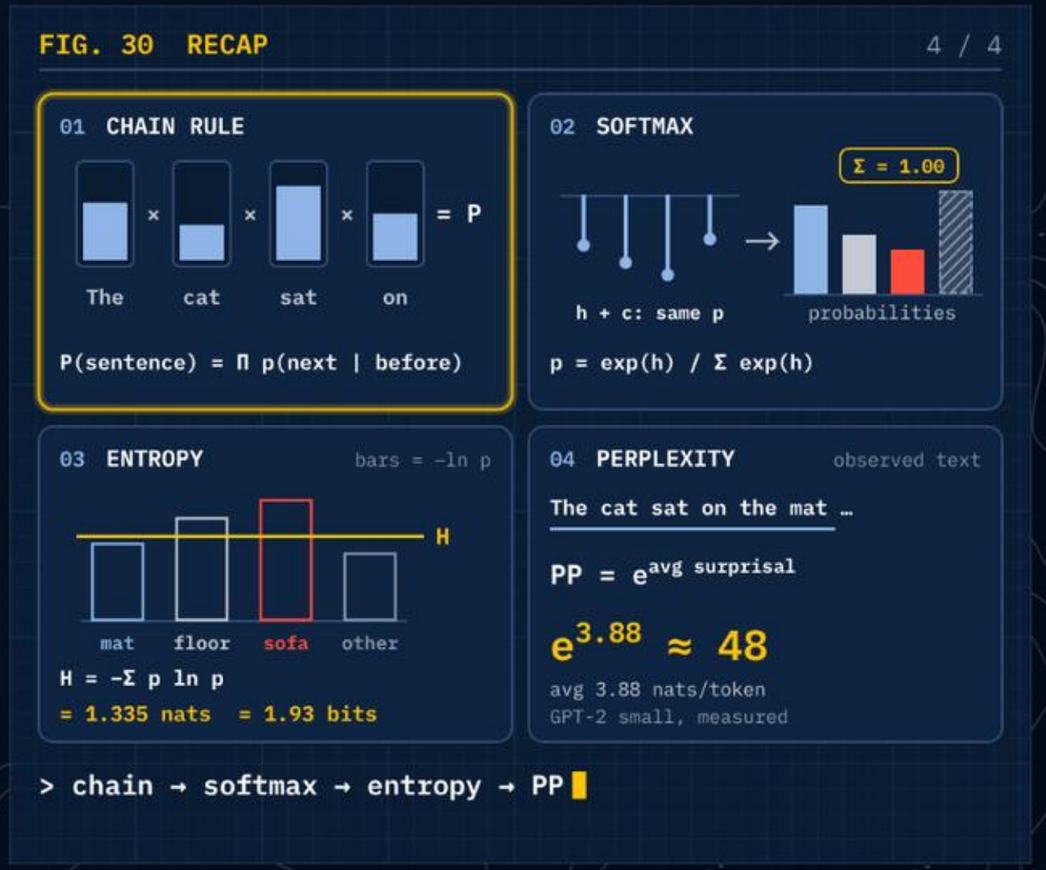
Kinvecta

NEXT TIME

Episode 5: Calculus for learning.
Nudge the score for mat.
Watch its surprisal fall.

In the comments: what surprised you more, the +5 that changes nothing, or 97% sure with a perplexity of 100?

How Large Language Models Work · subscribe for Episode 5



MARK